

TCP1P CTF Special Crismas 2025 Write Up

A. Letter Times

Kita diberikan dua bagian dari dataset, yaitu peraturan dan cipher. Peraturannya adalah sebuah daftar berisi nama Bahasa Inggris dari masing-masing angka dan nilai numeriknya, seperti one = 1, two = 2, sixty = 60, dst serta syarat di mana setiap variabel harus ≥ 0 . Cipher adalah sebuah block berisi string dengan panjang 6 huruf seperti sennsu, eeegnw, and ewveer. Tujuan dari masalah atau challenge ini adalah decode semua string.

Ide pertama saya adalah simulasi masalah sebagai Sistem Persamaan Linear (SPL) di mana masing-masing huruf dijumlahkan menjadi sebuah nilai (misalnya, o+n+e = 1). Saya membuat sebuah matriks untuk menyelesaikan unknown, tetapi hasilnya berantakan. Saya mendapatkan bilangan bulat negatif untuk beberapa nilai dari huruf yang melanggar aturan dari challenge ini. Kemudian, saya baca judul challenge, yaitu "Letter Times" dan hint berupa "Round". Hal ini mengubah strategi saya dari relasi penjumlahan menjadi perkalian.

Penyelesaian atas sistem perkalian lumayan rumit, tetapi kita bisa mengubah ini menjadi sistem linear dengan logaritma karena $\log(ab) = \log(a) + \log(b)$. Dengan logaritma natural dari nilai ($\ln(1)$, $\ln(2)$, dst), persamaan di peraturan menjadi linear ($\ln(o) + \ln(n) + \ln(e) = 0$)

Kita sebenarnya bisa melakukan ini secara manual (seperti ujian matematika). Namun, teknik ini memakan waktu dan perkalian desimal untuk >50 string cipher adalah suatu kegiatan yang "nguli". Jadi, saya memiliki ide untuk mencari cara untuk membuat script python untuk menyelesaikan masalah ini. Saya menggunakan python dengan numpy untuk menyelesaikan SPL logaritma dengan cara:

1. Petakan persamaan ke dalam matriks.
2. Selesaikan nilai log dari masing-masing huruf.
3. Ubah nilai log menjadi bilangan ril dengan $\exp()$.
4. Hitung perkalian dari huruf di dalam cipher string.

5. Bulatkan hasil ke bilangan bulat terdekat dan ubah ke dalam ASCII. ()

Kode A.1: Script Solver Challenge

```
import numpy as np

# 1. Setup the equations
# We will solve: log(letter1) + log(letter2) + ... = log(value)
equations = [
    ("one", 1), ("two", 2), ("three", 3), ("four", 4), ("five",
5),
    ("six", 6), ("seven", 7), ("eight", 8), ("nine", 9), ("ten",
10),
    ("twenty", 20), ("thirty", 30), ("forty", 40), ("fifty", 50),
("sixty", 60)
]

# Identify all unique letters
letters = sorted(list(set("".join([w for w, v in equations]))))
letter_map = {l: i for i, l in enumerate(letters)}
n_vars = len(letters)

# 2. Build the Linear System for Logarithms
A = np.zeros((len(equations), n_vars))
b = np.zeros(len(equations))

for row, (word, val) in enumerate(equations):
    # The value side is log(value). log(1) is 0.
    b[row] = np.log(val)
    for char in word:
        col = letter_map[char]
        A[row, col] += 1

# 3. Solve for log(letters) using Least Squares
# We use lstsq because the system might be overdetermined
log_x, residuals, rank, s = np.linalg.lstsq(A, b, rcond=None)

# Convert back to real values: letter = exp(log_x)
# We map them to a dictionary for easy use
letter_values = {l: np.exp(val) for l, val in zip(letters, log_x)}
```

```

# 4. Decode the Cipher
cipher_words = [
    "sennsu", "eeegnw", "ewveer", "vtfesg", "rexifr", "wsfwes",
    "xgtgee",
    "reerux", "eisifh", "enwege", "eetggx", "regvws", "vgeuge",
    "heisif",
    "fftfei", "eegfgf", "gtxgee", "etftve", "sverwg", "gveegu",
    "nggeug",
    "ieffgf", "eenesv", "eshiif", "fehiis", "uextst", "wrwuee",
    "revwsg",
    "ewvsrg", "iifhse", "wereev", "sxtsws", "xgeget", "gteuvg",
    "wuweer",
    "ettnen", "ggffee", "eggext", "ihiefs", "urweew", "erxrue",
    "hisfie",
    "sxeeeu", "egngwt", "gtxeeg", "vfteee", "eihsif", "tswtxi",
    "sefihi",
    "esvsve", "eegtgx", "wrieww", "ftevee", "vgeeug", "ersntv"
]

decoded_message = ""

for word in cipher_words:
    # "Letter Times": Multiply the values of the letters
    product = 1.0
    for char in word:
        if char in letter_values:
            product *= letter_values[char]

    # "Round": Round to the nearest integer for ASCII
    ascii_val = int(round(product))
    decoded_message += chr(ascii_val)

print(f"\nDecoded Message:\n{decoded_message}")

```

Setelah masalah di atas selesai, script akan menghasilkan sebuah teks berupa flag.

Flag: XMAS{G07_M0l3_pr08l3m5?__C4l1_Av094dr0_47_6.02_x_10^23}